CCAR-P · DOMAIN 3 · FOLD AND SAY

The D3 Recall Deck

Forty cards for the exact things the exam punishes

40 cards · 5 per objective · the five laws word for word · every canon number
Fold each page along the red line. Read the left side, say the right side aloud, then unfold and check.

Cards per objective



Prepared by Ravi Ranjan · CCAO-F, CCDV-F & CCAR-F

Built 13 Oct 2026 · facts verified against the live docs 13 Oct 2026

THREE PASSES, SPACED

How to use the deck

Fold
Fold the page on the red dashed line.

→

Say
Read the cue; say the answer aloud, whole.

→

Check
Unfold. Exact numbers or it is a miss.

→

Tick
Tick P1, P2 or P3 only when it was exact.

Pass	When	Bar
Pass 1	Sat 17 Oct, evening (15 min)	no bar — find the gaps
Pass 2	Mon 26 Oct, evening, after the D3 repair day	≥ 36 / 40
Pass 3	Sat 31 Oct, morning, before Mixed Mock II	≥ 38 / 40

The five D3 laws

1. Every tool you add is a door; know who holds the key.
2. Chunk along meaning, not length.
3. Retrieval follows the shape of the data, not the fashion.
4. You cannot fix what you cannot see.
5. Send the map first, the territory on demand.

01 · 3.1 Capability bloat

Every tool schema you carry costs you two things:

☐ P1 ☐ P2 ☐ P3 law 1

Tokens on every call, and wrong-wrench risk: lookalike tools confuse the model's tool selection.

02 · 3.1 Capability bloat

E2 bloat audit: nine tools' schemas per call, trimmed to the three used —

☐ P1 ☐ P2 ☐ P3

2,314 → **811** tokens per call. Saving **1,503** tokens × **40,000** calls a day = **60,120,000** tokens a day.

03 · 3.1 Capability bloat

Trim tools by usage logs alone? Add the second test:

☐ P1 ☐ P2 ☐ P3 law 1

Consequence. Keep a rarely used tool if missing it is costly, e.g. `escalate_to_human`. Trim by usage AND consequence.

04 · 3.1 Capability bloat

Two tools do the same job and the model keeps picking the wrong one. The fix:

☐ P1 ☐ P2 ☐ P3

Merge the lookalikes into one tool and give it a sharp description. The description is what the model chooses by.

05 · 3.1 Capability bloat

A subagent gets which tools?

☐ P1 ☐ P2 ☐ P3 law 1

Only the tools its job needs. Every extra schema is tokens and another door. Adding a tool IS right when the job genuinely needs it.

06 · 3.2 Authentication & authorization

D3 Law 1, word for word — the law about every tool you connect:

☐ P1 ☐ P2 ☐ P3 law 1

Every tool you add is a door; know who holds the key. An MCP server is a door too: whoever holds its token holds every tool it exposes.

07 · 3.2 Authentication & authorization

authN versus authZ — say both, with the dock picture:

☐ P1 ☐ P2 ☐ P3 law 1

authN = who you are (the gate pass). authZ = what you may do (the manifest of bays). A correctly signed-in caller can still hold too many bays.

08 · 3.2 Authentication & authorization

Where the authorization check must live, and what the audit log keeps:

☐ P1 ☐ P2 ☐ P3 law 1

In code, before the tool runs: allowlist by scope, not by prompt. A sign is not a lock. The audit log records denials as well as grants.

09 · 3.2 Authentication & authorization

Credentials for integrations — three rules:

☐ P1 ☐ P2 ☐ P3 law 1

Short-lived, scoped OAuth tokens over permanent shared API keys; a separate service account per integration; the model never sees or chooses credentials.

10 · 3.2 Authentication & authorization

Transport shapes authentication — stdio versus Streamable HTTP:

☐ P1 ☐ P2 ☐ P3 law 1

stdio: a local process on the same machine, inheriting the user's environment. Streamable HTTP: remote, so it needs real authentication (OAuth).

11 · 3.3 Accuracy–latency trade-offs

The justification sentence for a dial move has four parts:

☐ P1 ☐ P2 ☐ P3

Constraint + dial + direction + number. 'To meet the **4 s** p95 policy SLA we cut top-k from **40** to **20,** losing **0.3** pts recall@**20**.'

12 · 3.3 Accuracy–latency trade-offs

Streaming the reply cuts which latency — and which does it leave alone?

☐ P1 ☐ P2 ☐ P3

It cuts time-to-first-token only. Total generation time is unchanged, so streaming alone never rescues a total-time p95 SLA.

13 · 3.3 Accuracy–latency trade-offs

Tie-break when two configurations both clear the constraint — and when only one does:

☐ P1 ☐ P2 ☐ P3

Both clear it: take the cheaper/faster one. Only one clears it: the constraint decides, however fast or cheap the other looks.

14 · 3.3 Accuracy–latency trade-offs

Minimum cacheable prefix, by model tier — the full ladder:

☐ P1 ☐ P2 ☐ P3

512 tokens (Fable **5.1**, Opus **5.5**, Sonnet **5.5**, Haiku **5.5**) · **1,024** (Sonnet **4**.x) · **2,048** (Opus **4.7**) · **4,096** (Haiku **4.5**, Opus **4.5**/4.6). Below the minimum, nothing caches — silently.

15 · 3.3 Accuracy–latency trade-offs

The caching dial: write price, read price, breakpoint limit —

☐ P1 ☐ P2 ☐ P3

Writes **1.25×** (**5**-minute) or **2×** (**1**-hour). Reads **0.1×** most models, **0.05×** Opus/Sonnet **5.5**, **0.025×** Fable **5.1**. ≤ **4** breakpoints. Changing tools, thinking or effort invalidates the prefix.

16 · 3.4 Observability at scale

D3 Law 4, word for word — the observability law:

☐ P1 ☐ P2 ☐ P3 law 4

You cannot fix what you cannot see. Dashboards render signal; they do not create it. Alerts: few, tied to SLAs, actionable.

17 · 3.4 Observability at scale

Logs, traces, metrics — the question each one answers:

☐ P1 ☐ P2 ☐ P3 law 4

Logs: what happened, per event. Traces: where the time went for ONE request (one trace ID across every hop). Metrics: counters and histograms — p50/p95/p99, tokens, cost, errors per stage.

18 · 3.4 Observability at scale

E7 dock cameras: the mean read 310 ms, but…

☐ P1 ☐ P2 ☐ P3 law 4

p95 was **2,900 ms**. The mean hid the tail; one slow ticket's end-to-end trace, **4,100 ms**, showed which stage. SLAs are promises about tails, not means.

19 · 3.4 Observability at scale

The cardinality trap — what must never become a metric label?

☐ P1 ☐ P2 ☐ P3 law 4

Unbounded IDs (shipment ID, user ID): each new value makes a new series. Keep metric labels bounded; put the IDs in logs and traces.

20 · 3.4 Observability at scale

At 40,000 shipments a day you sample traces — but keep what, in full?

☐ P1 ☐ P2 ☐ P3 law 4

100% of errors. Sample the routine traces, never the failures: the failed requests are the ones you must follow end to end.

21 · 3.5 RAG pipeline: chunking & indexing

D3 Law 2, word for word — and what it means for heading-structured SOPs:

☐ P1 ☐ P2 ☐ P3 law 2

Chunk along meaning, not length. Heading-aware chunks with a little overlap; keep a table with its header. Fixed-size cut SOP **214** §**4** and retrieved the cancellation steps.

22 · 3.5 RAG pipeline: chunking & indexing

Contextual retrieval adds what, to each chunk, before which two steps?

☐ P1 ☐ P2 ☐ P3 law 2

A short, chunk-specific context ('This chunk is from SOP **214** §**4,** customs e-way bill re-filing…'), prepended before embedding AND before BM25 indexing.

23 · 3.5 RAG pipeline: chunking & indexing

The contextual-retrieval ladder: top-20 failure rate, step by step —

☐ P1 ☐ P2 ☐ P3

Baseline **5.7%** → contextual embeddings **3.7%** (−**35%**) → + contextual BM25 **2.9%** (−**49%**) → + reranking **1.9%** (−**67%**).

24 · 3.5 RAG pipeline: chunking & indexing

Retrieve how many, rerank, then pass how many to the model?

☐ P1 ☐ P2 ☐ P3

Retrieve broadly, about **150** chunks → rerank against the question → pass the top **20**. Adding the reranker stage IS sometimes the right move.

25 · 3.5 RAG pipeline: chunking & indexing

When should you skip RAG altogether — and what do you do instead?

☐ P1 ☐ P2 ☐ P3

When the whole knowledge base is under ~**200k** tokens: put it all in the prompt and cache it. Above that, build the retrieval pipeline.

26 · 3.6 Retrieval strategy

D3 Law 3, word for word — the retrieval law:

☐ P1 ☐ P2 ☐ P3 law 3

Retrieval follows the shape of the data, not the fashion. Exact codes → keyword-leaning; symptoms and paraphrases → semantic-leaning.

27 · 3.6 Retrieval strategy

Route by data shape: an ID, a count, a policy question go to… within what SLA?

☐ P1 ☐ P2 ☐ P3 law 3

ID → structured lookup, no RAG (≤ **2 s**). Count or sum → SQL-style aggregate, RAG cannot count (≤ **5 s**). Policy or how-to → hybrid RAG (≤ **4 s**).

28 · 3.6 Retrieval strategy

Hybrid search is two searches fused. Name both, and each one's blind spot:

☐ P1 ☐ P2 ☐ P3 law 3

BM25/keyword (exact codes, IDs, rare terms) + embeddings (meaning, paraphrase), scores fused, shortlist reranked. Keyword alone fails on paraphrase; semantic alone fails on exact codes.

29 · 3.6 Retrieval strategy

Recall, precision and context poisoning — retrieval sense:

☐ P1 ☐ P2 ☐ P3 law 3

Recall: did the right chunk make it into the retrieved set. Precision: how much of what was retrieved is relevant. Poisoning: wrong or stale chunks injected and trusted.

30 · 3.6 Retrieval strategy

Kaveri's mis-routed consignment-ID lookup, sent through RAG, cost…

☐ P1 ☐ P2 ☐ P3 law 3

₹1,800 a day against **₹4** for the right drawer, a structured lookup. The ID was a key, not a meaning.

31 · 3.7 Connection protocols

MCP's three primitives — name each and who invokes it:

☐ P1 ☐ P2 ☐ P3

Tools: model-invoked actions. Resources: application-provided data. Prompts: user-invoked templates.

32 · 3.7 Connection protocols

MCP's two current transports — and the one that is deprecated:

☐ P1 ☐ P2 ☐ P3

stdio (local) and Streamable HTTP (remote). The old HTTP+SSE transport is deprecated; on the exam, answer with stdio versus Streamable HTTP.

33 · 3.7 Connection protocols

How Claude Code and the Agent SDK name an MCP tool:

☐ P1 ☐ P2 ☐ P3 law 1

`mcp__<server>__<tool>`, e.g. `mcp__kaveri__get_shipment_status`. One server, one key: whoever holds its token holds every tool it exposes.

34 · 3.7 Connection protocols

Direct API/CLI, MCP or agent-to-agent — when does each win?

☐ P1 ☐ P2 ☐ P3

API/CLI: one fixed, stable call from your own code. MCP: many clients, many tools, reuse across apps — write the server once. Agent-to-agent: the other side owns its own goals and tools.

35 · 3.7 Connection protocols

MCP spec 2026-07-28 drift — what changed?

☐ P1 ☐ P2 ☐ P3

Stateless: every request carries its own `_meta`; `server/discover`; sampling, logging and roots deprecated (**12**-month window), dynamic client registration too; stdio + Streamable HTTP; elicitation current.

36 · 3.8 Progressive vs monolithic context

D3 Law 5, word for word — the context-strategy law:

☐ P1 ☐ P2 ☐ P3 law 5

Send the map first, the territory on demand. Progressive discovery: a catalogue of names + one-line descriptions, full schemas or documents fetched on demand. Monolithic: everything, every call.

37 · 3.8 Progressive vs monolithic context

E8 catalogue at the gate: monolithic versus progressive, tokens per call —

☐ P1 ☐ P2 ☐ P3 law 5

362,460 vs **4,205** ≈ **86×**. **4,205** = **9** stubs × **25** + **260** header + **20** summaries × **180** + one **120**-token fetch. (The first print's **4,625** / **78×** is wrong.)

38 · 3.8 Progressive vs monolithic context

Where monolithic beats progressive discovery — the crossover:

☐ P1 ☐ P2 ☐ P3 law 5

A small, stable, always-used set (≈ **4,500** tokens) is better sent whole and cached. Progressive wins for large, changing, rarely used sets.

39 · 3.8 Progressive vs monolithic context

The API-native catalogue for tools: which tool, which flag?

☐ P1 ☐ P2 ☐ P3 law 5

The tool search tool (`tool_search_tool_regex_20251119` or `tool_search_tool_bm25_20251119`, GA) with `defer_loading: true` on the tools held back.

40 · 3.8 Progressive vs monolithic context

Context editing `clear_tool_uses_20250919`: default trigger and what it keeps —

☐ P1 ☐ P2 ☐ P3 law 5

Triggers at ~**100k** input tokens by default and keeps the **3** most recent tool uses; tune with `clear_at_least` and `exclude_tools`. Beta.